

1. Getting Started with MySQL Stored Procedures

Writing the first stored procedure

The first stored procedure is very simple. It retrieves all products from *products* table. First let's take a look at the stored procedure source code below:

```
1 DELIMITER //
2 CREATE PROCEDURE GetAllProducts()
3 BEGIN
4 SELECT * FROM products;
5 END //
6 DELIMITER ;
```

Let's examine the stored procedure above in a great details:

- The first command you see is DELIMITER //. This command is not related to the stored procedure. DELIMITER statement in MySQL is used to change the standard delimiter (semicolon) to another. In this case, the delimiter is changed from semicolon(;) to //, so you can have multiple SQL statements inside stored procedure which can be separated by the semicolon. After the END keyword we use delimiter // to show the end of the stored procedure. The last command changes the delimiter back to the standard one (semicolon).
- In order to create a new stored procedure you use CREATE PROCEDURE statement. After the CREATE PROCEDURE statement you can specify the name of stored procedure. In this case, the stored procedure name is *GetAllProducts*.
- Everything inside a pair of keyword BEGIN and END is called store procedure's body. You can write declarative SQL code in the stored procedure's body.

Now we have created a new stored procedure. It is time to know how to invoke in command line of MySQL.

Calling the stored procedure

In order to invoke a stored procedure, we use the following SQL command:

```
1 CALL STORED_PROCEDURE_NAME()
```

First you use keyword CALL followed by the stored procedure name and a pair of parenthesis. To invoke the stored procedure *GetAllProducts*, we use the following command:

```
1 CALL GetAllProducts();
```

If you run the command above you will get all products in the *products* database table.

2. Variables in Stored Procedures

Summary: in this tutorial you will learn about variables in [MySQL stored procedure](#), how to declare and use variables. In addition, scope of variable is also covered

Declaring variables

Variables are used in [stored procedure](#) to store the immediate result. You can declare a variable by the following syntax:

[Type text]

```
1 DECLARE variable_name datatype(size) DEFAULT default_value;
```

Followed DECLARE statement is the variable name. The variable name should follow the naming convention and should not be the same name of table or column in a database. Next you can specify the data type of the variable, it can be any primitive type which MySQL supports such as INT, VARCHAR and DATETIME...along with the data type is the size of the variable. When you declare a variable, its initial value is NULL. You can also assign the default value for the variable by using DEFAULT statement. For example, we can define a variable name total_sale with the data type INT and default value is 0 as follows:

```
1 DECLARE total_sale INT DEFAULT 0
```

To declare two or more variables with the same data type we can use only just one DECLARE such as:

```
1 DECLARE x, y INT DEFAULT 0
```

We declared two variables x and y with the same data type INT and their default value is zero.

Assigning variables

Once you declared a variable, you can start using it. To assign other value to a variable you can use SET statement, for example:

```
1 DECLARE total_count INT DEFAULT 0
```

```
2 SET total_count = 10;
```

The total_count variable's value now is ten (10).

Besides SET statement, we can use [SELECT INTO statement](#) to assign result of a query to a variable.

```
1 DECLARE total_products INT DEFAULT 0
```

```
2 SELECT COUNT(*) INTO total_products
```

```
3 FROM products
```

In the example above, we declare a variable total_products and initialize its value to zero. Then we use SELECT ... INTO statement to assign the variable total_products with the total products in products database table.

Variables scope

A variable has its own scope. If you declare a variable inside a stored procedure, it will be out of scope when the END of stored procedure reached. If you defined a variable inside block BEGIN/END inside a stored procedure it will be out of scope if the END reached. You can declare two variables or more variables with the same name in different scopes; the variable only is effective in its scope.

A variable with the '@' at the beginning is session variable. It exists until the session end.

3. MySQL Stored Procedure Parameters

Summary: In this tutorial, you will learn how to write MySQL stored procedures with parameters. We will also give you a couple of stored procedure examples to help you understand more about using parameters in stored procedures.

Almost stored procedures you develop require parameters. Parameters make the stored procedure more flexible and useful. In MySQL, a parameter has one of three modes **IN**, **OUT** and **INOUT**.

- **IN** this is the default mode. IN indicates that a parameter can be passed into stored procedures but any modification inside stored procedure does not change parameter. Suppose you pass parameter *Id*, which is equal 10, into stored procedure *GetAll(Id)*, after executing the stored procedure the value of *Id* is still 10 even though the *GetAll* stored procedure can change the value of it.

- **OUT** this mode indicates that stored procedure can change this parameter and pass back to the calling program.
- **INOUT** obviously this mode is combined of IN and OUT mode; you can pass parameter into stored procedure and get it back with the new value from calling program.

The syntax of defining a parameter in stored procedure is as follows:

MODE param_name param_type(param_size)

MODE could be IN, OUT or INOUT depending on the purpose of parameter you specified.

param_name is the name of the parameter. The name must not be the same as the column name of tables and following naming convention. Followed the parameter name is the type of parameter and its size.

Each parameter is separated by a comma if the stored procedure more than one parameter.

Let's practice with following examples to understand more.

The **first example** is a stored procedure to get all offices in a country. Here is the SQL source code:

```
1 DELIMITER //
2 CREATE PROCEDURE GetOfficeByCountry(IN countryName VARCHAR(255))
3 BEGIN
4     SELECT city, phone
5     FROM offices
6     WHERE country = countryName;
7 END //
8 DELIMITER ;
```

As you can see we use countryName as the IN parameter with its type is varchar and its size is 255. In body part of stored procedure, we retrieve all offices which its country is the *countryName*.

Suppose you want to retrieve all office in USA, just pass the value to the stored procedures like this:

```
1 CALL GetOfficeByCountry('USA')
```

It returns all offices in USA

To get all offices in France just pass France to the stored procedure as follows:

```
1 CALL GetOfficeByCountry('France')
```

It returns all offices in France.

The **second example**, we will write a store procedure to count the order in a specific order status such as shipped, resolved, cancelled, on hold, disputed or in process. Here is the stored procedure

```
01 DELIMITER $$
02 CREATE PROCEDURE CountOrderByStatus(
03     IN orderStatus VARCHAR(25),
04     OUT total INT)
05 BEGIN
06     SELECT count(orderNumber)
07     INTO total
08     FROM orders
09     WHERE status = orderStatus;
10 END$$
11 DELIMITER ;
```

[Type text]

The CountOrderByStatus stored procedure has two parameters:

- orderStatus parameter is **IN** parameter; we pass order status such as shipped or on hold in to get the number of it
- total parameter is the **OUT** parameter which we use to get the total order by a specified status back.

So to get number of shipped orders, we just perform following statements

```
1 CALL CountOrderByStatus('Shipped',@total);
```

To get number of in process we do the same as above

```
1 CALL CountOrderByStatus('in process',@total);
```

```
2 SELECT @total AS total_in_process;
```

In the third procedure, we will demonstrate the **INOUT** parameter. The stored procedure capitalizes all words in a string and returns it back to the calling program. The stored procedure source code is as follows:

```
01 DELIMITER $$
02 CREATE PROCEDURE `Capitalize`(INOUT str VARCHAR(1024))
03 BEGIN
04     DECLARE i INT DEFAULT 1;
05     DECLARE myc, pc CHAR(1);
06     DECLARE outstr VARCHAR(1000) DEFAULT str;
07     WHILE i <= CHAR_LENGTH(str)
08     DO
09         SET myc = SUBSTRING(str, i, 1);
10         SET pc = CASE WHEN i = 1 THEN '
11         ELSE SUBSTRING(str, i - 1, 1)
12         END;
13         IF pc IN (' ', '&', '"', '_', '?', ',', ':', '!', ';', '-', '/', '(', ')') THEN
14             SET outstr = INSERT(outstr, i, 1, UPPER(myc));
15         END IF;
16         SET i = i + 1;
17     END WHILE;
18     SET str = outstr;
19 END$$
20 DELIMITER ;
```

Here is the usage of the Capitalize stored procedure

```
1 SET @str = 'mysql stored procedure tutorial';
```

```
2 CALL Capitalize(@str);
```

```
3 SELECT @str;
```

And the @str value is 'Mysql Stored Procedure Tutorial'

4. Conditional Control in MySQL Stored Procedure

Conditional control enables you to execute the code based on the value of an expression or a combination of expression using logical operators. MySQL supports two conditional control statements such as IF and CASE.

The IF Statement

The syntax of IF statement is simple as follows:

```
1 IF expression THEN commands
2 [ELSEIF expression THEN commands]
3 [ELSE commands]
4 END IF;
```

The *commands* associated with IF or ELSEIF or ELSE only executed when the expression is evaluated as TRUE. One of the common trap of IF statement is NULL value; When the expression is evaluated as NULL it is neither TRUE nor FALSE. Actually NULL is unknown value.

Here are several combinations of the IF statement.

```
1 IF expression THEN commands
```

```
2 END IF;
```

```
1 IF expression THEN commands
```

```
2 ELSE commands
```

```
3 END IF;
```

```
1 IF expression THEN commands
```

```
2 ELSEIF expression THEN commands
```

```
3 ELSE commands
```

```
4 END IF;
```

It is important to note that you can have an IF statement inside another IF statement

The CASE Statement

When multiple conditions are used with IF statement the code is not easy to read and maintain. In this case, the CASE statement can be used to make the code easier to read and understand. The following illustrates the syntax of the CASE statement:

```
1 CASE case_expression
2 WHEN when_expression THEN commands
3 WHEN when_expression THEN commands
4 ...
5 ELSE commands
6 END CASE;
```

In the CASE syntax above, *case_expression* can be any valid expression. The *case_expression*'s value is used to compare with *when_expression* in each WHEN clause for equality. If an equal *when_expression* is found, the *commands* in that after the THEN clause of the corresponding WHEN clause executes. The ELSE clause is optional and used in case no equal WHEN clause found.

5. Loop in Stored Procedures

Summary: *In this tutorial, you will learn how to use various loop statements including WHILE, REPEAT and LOOP to run a block of code repeatedly in MySQL.*

MySQL stored programming language supports loop which allows you to process commands iteratively. The standard loops are discuss as follows

WHILE loop

The syntax of while loop is as follows:

```
1 WHILE expression DO
2   Statements
3 END WHILE
```

First the *while loop* checks the expression, if it is true it will executes statement until the expression become false. Because *while loop* checks the expression before *statements* executed, it is often known as pretest loop. Here is an example of using *while loop* in stored procedure:

```
01 DELIMITER $$
02 DROP PROCEDURE IF EXISTS WhileLoopProc$$
03 CREATE PROCEDURE WhileLoopProc()
04   BEGIN
05     DECLARE x INT;
06     DECLARE str VARCHAR(255);
07     SET x = 1;
08     SET str = "";
09     WHILE x <= 5 DO
10       SET str = CONCAT(str,x,',');
11       SET x = x + 1;
12     END WHILE;
13     SELECT str;
14   END$$
15 DELIMITER ;
```

In stored procedures above, we build string repeatedly until the variable x greater than 5 and then we output the built string into console screen by using SELECT statement. One of common trap almost developers encounter is if the variable x is not initialized, its default value is NULL so the condition in while loop is always true; the code block inside while loop is executed indefinitely until your database server crashed.

REPEAT loop

The syntax of repeat loop is as follows:

```
1 REPEAT
2   Statements;
3 UNTIL expression
4 END REPEAT
```

First the *statements* are executed, and then the *expression* is evaluated. If the *expression* is evaluated as true the statements are executed again and again until its value become false. Because the *repeat loop* checks the *expression* after the execution of *statements* so it is also known as post-test loop. We can rewrite the stored procedure above by using repeat loop as follows:

```
01 DELIMITER $$
02 DROP PROCEDURE IF EXISTS RepeatLoopProc$$
03 CREATE PROCEDURE RepeatLoopProc()
04   BEGIN
05     DECLARE x INT;
```

```

06 DECLARE str VARCHAR(255);
07 SET x = 1;
08 SET str = "";
09 REPEAT
10     SET str = CONCAT(str,x,',');
11     SET x = x + 1;
12 UNTIL x > 5
13 END REPEAT;
14 SELECT str;
15 END$$
16 DELIMITER ;

```

Be noted that there is no delimiter (;) after *UNTIL expression*

LOOP loop, LEAVE and ITERATE

Leave statement allows you to leave the loop. It is a bit like *break* in other languages such as Java, C#...

Iterate statement allows you to start the loop again. It is like *continue* in Java or C#.

MySQL also supports a LOOP loop which allows you to execute statements repeatedly and more flexible. Here is an example of using LOOP loop.

```

01 DELIMITER $$
02 DROP PROCEDURE IF EXISTS LOOPLoopProc$$
03 CREATE PROCEDURE LOOPLoopProc()
04 BEGIN
05     DECLARE x INT;
06     DECLARE str VARCHAR(255);
07     SET x = 1;
08     SET str = "";
09     loop_label: LOOP
10         IF x > 10 THEN
11             LEAVE loop_label;
12         END IF;
13         SET x = x + 1;
14         IF (x mod 2) THEN
15             ITERATE loop_label;
16         ELSE
17             SET str = CONCAT(str,x,',');
18         END IF;
19     END LOOP;
20
21     SELECT str;
22 END$$
23 DELIMITER ;

```

The stored procedure only constructs string with even numbers. First we defined a loop label, if a variable X is greater than 10 the loop is ended because of the LEAVE statement. If the variable X is odd, the ITERATE statement ignores everything below it and continues. If the variable X is even, the block after ELSE constructs strings with even numbers.

SQL Cursor in Stored Procedures

Summary: *In this tutorial, you will learn how to use database cursor in stored procedure to loop over a result set in MySQL.*

MySQL supports cursor in stored procedures, functions and triggers. Cursor is used to iterate through a set of rows, which returned by a query, and process individual row. Currently with all versions greater 5.x, MySQL cursor has following properties:

- Read only: it means you cannot update the cursor.
- Non-scrollable: it only can traverse in one direction and cannot skip, move back or forth in result set.
- Asensitive: you should avoid update table while open a cursor on that table otherwise you may get unexpected results.

MySQL supports following statements for working with cursor.

First you have to declare a cursor using DECLARE statement:

```
1 DECLARE cursor_name CURSOR FOR SELECT_statement;
```

Second you have to open the cursor using OPEN statement. You must open cursor before fetching rows from it.

```
1 OPEN cursor_name;
```

Next you can retrieve next row from cursor and move the cursor to the following row in a result set by using FETCH statement.

```
1 FETCH cursor_name INTO variable list;
```

And finally, you must close the cursor to deactivate it and release the memory associated with that cursor. To close the cursor you use CLOSE statement:

```
1 CLOSE cursor_name;
```

One of the most important point when working with cursor is you should use a NOT FOUND handler to avoid raising a fatal "no data to fetch" condition.

We use a stored procedure example below to demonstrate cursor.

```
01 DELIMITER $$
02 DROP PROCEDURE IF EXISTS CursorProc$$
03 CREATE PROCEDURE CursorProc()
04 BEGIN
05     DECLARE no_more_products, quantity_in_stock INT DEFAULT 0;
06     DECLARE prd_code VARCHAR(255);
07     DECLARE cur_product CURSOR FOR
08         SELECT productCode FROM products;
09     DECLARE CONTINUE HANDLER FOR NOT FOUND
10         SET no_more_products = 1;
11
12     /* for logging information */
13     CREATE TABLE infologs (
14         Id int(11) NOT NULL AUTO_INCREMENT,
15         Msg varchar(255) NOT NULL,
```

[Type text]

```
16 PRIMARY KEY (Id)
17 );
18 OPEN cur_product;
19
20 FETCH cur_product INTO prd_code;
21 REPEAT
22     SELECT quantityInStock INTO quantity_in_stock
23     FROM products
24     WHERE productCode = prd_code;
25
26     IF quantity_in_stock < 100 THEN
27         INSERT INTO infologs(msg) VALUES (prd_code);
28     END IF;
29
30     FETCH cur_product INTO prd_code;
31     UNTIL no_more_products = 1
32     END REPEAT;
33
34     CLOSE cur_product;
35
36     SELECT * FROM infologs;
37 DROP TABLE infologs;
38 END$$
39 DELIMITER;
```

The stored procedure is very simple and can archive the same result by SQL query. We use it only for demonstrating how cursors work.

We use a cursor for *products* table and loop though the products result set. If the quantity in stock of a product is less than 100, we log it into to a temporary table and after the loop we select all products to print it on screen.

Remember you must declare cursor first and then declare a NOT FOUND handler; otherwise you will get an error.

Below is an example of stored procedure to increase the loan amount percentage you want throw parameter. This example form tbl_loan.

[Type text]

```
DELIMITER //
CREATE PROCEDURE sp_increaseLoanAmount(IN percent INT)
BEGIN
    DECLARE existingAmount, newAmount, noMoreItem INT DEFAULT 0;
    DECLARE loanNo varchar(4);
    DECLARE cur_loanNo CURSOR FOR SELECT loan_num FROM tbl_loan;
    DECLARE CONTINUE HANDLER FOR NOT FOUND
        SET noMoreItem = 1;
    OPEN cur_loanNo;
    FETCH cur_loanNo INTO loanNo;
    REPEAT
        SELECT loan_amount into existingAmount from tbl_loan where loan_num = loanNo;
        SET newAmount = existingAmount + (existingAmount*percent)/100;
        UPDATE tbl_loan SET loan_amount = newAmount WHERE loan_num = loanNo;
        FETCH cur_loanNo INTO loanNo;
    UNTIL noMoreItem=1
    END REPEAT;
    CLOSE cur_loanNo;
END //
DELIMITER;
```

source of the document:

<http://www.mysqltutorial.org/mysql-stored-procedure-tutorial.aspx>

<http://www.mysqltutorial.org/mysql-triggers.aspx>